

AL-AZHARI LIBRARY — PHASE 13 IMPLEMENTATION REPORT

1. Status

Phase 13 complete

2. Exact Verification

- **TypeScript** (`npx tsc --noEmit`): PASS (0 errors)
- **ESLint** (`npm run lint`): PASS (0 errors, 0 warnings)
- **Build** (`npm run build`): PASS (Clean compilation via `tsc`)
- **Test Suites**: 100 / 100 PASS
- **Tests**: 729 / 729 PASS
- **Failures**: 0

3. Requirement Matrix

Requirement	Description	Status	Verification
NOT-001	The system SHALL provide in-app notifications for the order, payment, service, and pre-order lifecycle events listed in Section 11 (Notification Matrix).	COMPLETE	notification-api.test.ts , notification-transaction.test.ts , notification-schema.test.ts
NOT-002	The system SHALL send email for account verification, password reset, and other relevant transactional communication.	COMPLETE	email-adapter.test.ts
NOT-003	The system SHALL NOT rely on WhatsApp as an automated notification channel; WhatsApp remains a manually-initiated communication channel.	COMPLETE	Preserved across all services; zero automated WhatsApp webhooks or delivery hooks implemented.
NOT-004	Real-time-eligible events (new order, new service request, payment proof submitted, order status change) SHALL be pushed via Socket.IO to relevant Admin/Owner sessions and to the affected customer's session where applicable.	COMPLETE	realtime-socket.test.ts
RT-001	The system SHALL push real-time updates for: new order, new service request, payment proof submitted, order status change, and relevant Admin/customer updates.	COMPLETE	realtime-socket.test.ts
RT-002	The database SHALL remain the authoritative source of truth; real-time delivery is a notification mechanism only, not a data store.	COMPLETE	notification-transaction.test.ts , realtime-socket.test.ts
RT-003	If a real-time connection is unavailable, the underlying state change SHALL still be persisted and visible on next page load/refresh.	COMPLETE	notification-transaction.test.ts , email-adapter.test.ts

4. Files Created

1. src/modules/notifications/utils/notification.projection.ts — Safe customer notification projection stripping technical delivery metadata.
2. src/integrations/email/email.types.ts — Email service interface, template enums, and variables contract.
3. src/integrations/email/email.templates.ts — Bilingual (ar / en) transactional email template renderers.
4. src/integrations/email/email.service.ts — Nodemailer transporter implementation with testing capture stub and variable sanitization.
5. src/integrations/email/index.ts — Email integration exports.
6. src/realtime/events/socket.events.ts — Namespaced Socket.IO events and payload sanitizer.
7. src/realtime/socket/socket.auth.ts — Handshake JWT verification, session version validation, suspension checks.
8. src/realtime/socket/socket.rooms.ts — Room boundary authorization with zero existence leakage.
9. src/database/migrations/scripts/20260928_011_notifications.migration.ts — Versioned idempotent migration for notifications collection and indexes.
10. tests/unit/notification-schema.test.ts — Zod query validation tests.
11. tests/unit/email-adapter.test.ts — Email service tests (bilingual templates, secret suppression, deduplication).

12. `tests/unit/notifications-migration.test.ts` — Migration idempotency and rollback tests.
13. `tests/api/notification-api.test.ts` — Customer REST API endpoints and ownership isolation tests.
14. `tests/integration/realtime-socket.test.ts` — Socket.IO handshake auth, session revocation, room boundaries, permissions, and event emission tests.
15. `tests/integration/notification-transaction.test.ts` — Atomic transaction commit/rollback and deterministic dedupe enforcement tests.
16. `Back-end/docs/PHASE-13-NOTIFICATIONS-REALTIME.md` — Comprehensive technical documentation.
17. `docs/PHASE-13-NOTIFICATIONS-REALTIME.md` — Documentation copy in root docs.

5. Files Modified

1. `src/common/errors/errorCodes.ts` : Added stable Phase 13 error codes (`NOTIFICATION_NOT_FOUND` , `NOTIFICATION_OWNERSHIP_DENIED` , `SOCKET_AUTHENTICATION_FAILED` , `SOCKET_UNAUTHORIZED_ROOM` , `EMAIL_DELIVERY_FAILED`).
2. `src/modules/notifications/types/notification.types.ts` : Defined authoritative notification types (16 lifecycle events), channels, interfaces, and DTOs.
3. `src/modules/notifications/models/notification.model.ts` : Defined Mongoose schema with `strict: "throw"` , `timestamps: true` , and index constraints with `partialFilterExpression` .
4. `src/modules/notifications/models/outbox-event.model.ts` : Standardized partial index for deterministic dedupeKey.
5. `src/modules/notifications/repositories/notification.repository.ts` : Implemented repository queries with pagination, unread-first sorting, unread count, and atomic mark-as-read.
6. `src/modules/notifications/repositories/outbox-event.repository.ts` : Implemented outbox event repository.
7. `src/modules/notifications/schemas/notification.schema.ts` : Defined Zod input schemas for query pagination and ID params.
8. `src/modules/notifications/services/notification.service.ts` : Implemented atomic `recordLifecycleNotification` (persisting notification + pending outbox event), user queries, and read-state management.
9. `src/modules/notifications/controllers/notification.controller.ts` : Implemented REST controllers with ownership isolation.
10. `src/modules/notifications/routes/notification.routes.ts` : Bound customer routes under `/api/v1/notifications` .
11. `src/modules/notifications/index.ts` : Exported notification public module APIs.
12. `src/realtime/index.ts` : Initialized Socket.IO auth and room handlers via `initRealtime(io)` .
13. `src/app.ts` : Mounted notification routes at `/api/v1/notifications` .
14. `src/server.ts` : Attached `initRealtime(io)` on server bootstrap.
15. `src/database/migrations/index.ts` : Registered migration `20260928_011_notifications` .
16. `Back-end/ARCHITECTURE.md` : Documented Phase 13 contracts and architectural boundaries.

6. Migration

- **Filename:** `src/database/migrations/scripts/20260928_011_notifications.migration.ts`
- **Migration ID:** `20260928_011_notifications`
- **Target Collection:** `notifications`
- **Indexes Created:**
 - i. `{ recipientUserId: 1, createdAt: -1 }` (name: `idx_notifications_user_created`)
 - ii. `{ recipientUserId: 1, readAt: 1, createdAt: -1 }` (name: `idx_notifications_user_unread`)
 - iii. `{ dedupeKey: 1 }` with `{ unique: true, partialFilterExpression: { dedupeKey: { $type: "string" } } }` (name: `idx_notifications_dedupe`)
- **Idempotency & Safety:**
 - `up()` checks for existing collection and creates indexes safely without duplicating them.

- `down()` drops specific indexes non-destructively without dropping collections or deleting business data.

7. API Routes

Mounted under `/api/v1/notifications` (Protected via `authenticate`):

- `GET /api/v1/notifications` — List current user's notifications (paginated, unread-first sorting).
- `GET /api/v1/notifications/unread-count` — Count total unread notifications for current user.
- `GET /api/v1/notifications/:id` — Get single notification with ownership isolation.
- `PATCH /api/v1/notifications/:id/read` — Mark notification as read.
- `POST /api/v1/notifications/mark-all-read` — Mark all unread notifications of current user as read.

8. Socket Events & Rooms

- **Handshake Authentication:**
 - Verified against JWT access token (`socket.handshake.auth.token` OR `Authorization: Bearer <token>`).
 - Strict rejection of suspended accounts or revoked sessions (`tokenVersion` mismatch).
- **Authorized Rooms:**
 - `user:{userId}` — Auto-joined for personal notifications.
 - `order:{orderId}` — Customer owner or Admin/Owner with `orders.read` permission.
 - `service:{serviceId}` — Customer owner or Admin/Owner with `services.read` permission.
 - `admin:operational` — Admin/Owner users with operational permissions (`orders.read` , `payments.verify` , or `services.read`).
 - **Zero Leakage:** Unauthorized join attempts return generic forbidden responses without confirming record existence.
- **Namespaced Events:**
 - `notification.created`
 - `order.status_changed`
 - `payment.proof_submitted`
 - `service.request_created`
 - `service.quotation_sent`
 - `admin.queue_changed`
- **Payload Sanitization:** Strips passwords, tokens, payment proof URLs, sensitive account details, and private PII.

9. Email Adapter

- **Adapter:** `EmailServiceImpl` implementing `EmailService` .
- **Supported Templates:**
 - `verify_email`
 - `password_reset`
 - `order_confirmation`
 - `payment_update`
 - `service_update`
- **Locales:** Arabic (`ar`) and English (`en`) with HTML and plain text rendering.
- **Configuration:**
 - Uses `SMTP_HOST` , `SMTP_PORT` , `SMTP_SECURE` , `SMTP_USER` , `SMTP_PASS` , `EMAIL_FROM` .
 - In `test` mode, captures sent messages into an in-memory `sentEmails` array.

- **Failure Behavior:**
 - Email failures do not roll back business operations. Delivery failure is isolated to the delivery/outbox level.

10. Outbox Integration

- **Collection:** `outboxEvents`
- **Aggregate Types:** `"order" | "payment" | "service" | "user" | "return"`
- **Deterministic Dedupe Keys:**
 - `order:<orderId>:status:<status>:<version>`
 - `payment:<paymentId>:status:<status>:<version>`
 - `service:<serviceRequestId>:status:<status>:<version>`
 - `return:<returnRequestId>:status:<status>:<version>`
 - `refund:<refundId>:status:completed:<version>`
 - `user:<userId>:registered`
 - `user:<userId>:email_verified`
- **Transaction Boundaries:**
 - Business mutation, notification persistence, and outbox event creation execute inside the SAME MongoDB transaction.
 - Initial outbox event status is `"pending"` .
 - No network calls (SMTP, Socket.IO) are invoked inside MongoDB transactions.

11. Tests

Phase 13 includes 6 dedicated test suites (40 tests, all passing):

1. `tests/unit/notification-schema.test.ts` : 4 tests (Zod validation, query schema, ObjectId coercion).
2. `tests/unit/email-adapter.test.ts` : 4 tests (Email service contract, bilingual rendering, secret sanitization, dedupe tracking).
3. `tests/unit/notifications-migration.test.ts` : 3 tests (Migration idempotency, non-destructive down, index verification).
4. `tests/api/notification-api.test.ts` : 9 tests (Customer listing, unread count, ownership isolation, mark-as-read, pagination).
5. `tests/integration/realtime-socket.test.ts` : 15 tests (Handshake auth, session revocation, room boundaries, permission gating, payload sanitization).
6. `tests/integration/notification-transaction.test.ts` : 5 tests (Atomic transaction commit, rollback guarantees, duplicate prevention).

12. Open Decisions

- **OD-19 (Operational notification and email policy details):**
 - Strictly preserved.
 - No arbitrary recipient rules, retention policies, marketing emails, or invented delivery SLAs were introduced.
 - Email and realtime dispatches are triggered strictly through configuration and authoritative lifecycle events.

13. Regression

- All prior phases (Phases 0 through 12) remain 100% green.
- **Total Test Suites:** 100 passed / 100 total
- **Total Tests:** 729 passed / 729 total
- **Failures:** 0

14. Scope Boundary

No Phase 14 worker/retry/cron implementation was added.
No Phase 15 Reports work was added.
No Phase 16 Full Testing work was added.
No Phase 17 Security Hardening work was added.
No Phase 18 Deployment work was added.

15. Final Stop Condition

Phase 13 implementation is complete and verified. Execution is stopped here.